Suelen buscar que la **función greedy** no sea solo "el valor más alto", sino que **anticipe** el cumplimiento de las restricciones difíciles (como el equilibrio de hombres/mujeres en el examen 1 o el ratio de alumnos en el examen 3).

Para el **BRKGA**, la clave es entender que el cromosoma (un vector de números aleatorios entre 0 y 1) sirve para que el **Decoder** tome decisiones. Normalmente se usa de dos formas:

1. **Como orden de prioridad:** Ordenas los elementos según su valor en el cromosoma antes de pasarlos al greedy.
2. **Como sesgo (bias):** Multiplicas el valor de la función greedy por el valor del cromosoma para "explorar" opciones que no son las óptimas locales.

Aquí tienes 5 problemas diseñados siguiendo la estructura exacta de tus exámenes:

Problema 1: Asignación de Proyectos de Impacto Social

Una ONG tiene n voluntarios y m proyectos. Cada proyecto j tiene una capacidad mínima L_j y máxima U_j de voluntarios. Se conoce la afinidad a_{ij} del voluntario i con el proyecto j. Además, para que los proyectos sean diversos, el número de voluntarios con experiencia (set E) no puede superar al número de voluntarios sin experiencia (set S) en cada proyecto. El objetivo es maximizar la afinidad total.

Variables: $x_{ij} = 1$ si el voluntario i se asigna al proyecto j.

Heurística:

1. **Greedy:**
   - Define el conjunto de candidatos.
   - Diseña una función $q(i, j, P)$ que priorice a los voluntarios sin experiencia cuando un proyecto está cerca de romper el ratio de diversidad.
   - Escribe el pseudocódigo.
2. **BRKGA:**
   - Define la estructura del cromosoma.
   - Diseña un Decoder que utilice el cromosoma para determinar el orden en que los voluntarios eligen proyecto.

Resolución Problema 1: Proyectos de Impacto Social

Restricciones clave: Capacidad $[L_j, U_j]$ y diversidad (Expertos $E \leq$ Sin experiencia S).

(a) Greedy:

- **Candidatos:** Pares $\langle i, j \rangle$ donde el voluntario i aún no tiene proyecto y el proyecto j tiene hueco.
- **Función Greedy:**

$$q(i,j,P) = \begin{cases} -1 & \text{si } |P_j| \geq U_j \\ -1 & \text{si } i \in E \text{ y } |P_j \cap E| + 1 > |P_j \cap S| \\ a_{ij} + \text{peso_minimo}(j) & \text{en otro caso} \end{cases}$$

  Donde `peso_minimo(j)` da un bono extra si el proyecto j aún no ha llegado a su capacidad mínima L_j para incentivar que se llenen los proyectos.
- **Pseudocódigo:**
  i. $V_{pend} \leftarrow \{1..n\}, S \leftarrow \emptyset, \text{contadores } |P_j \cap E|, |P_j \cap S|$ a 0.
  ii. Mientras $V_{pend} \neq \emptyset$:
  iii. $\text{Candidatos } C = \{\langle i, j \rangle : i \in V_{pend}, j \in 1..m\}$.
  iv. $\text{Elegir } \langle i^*, j^* \rangle = \text{argmax } q(i, j, S)$.
  v. $\text{Si } q(i^*, j^*) = -1$ devolver INFEASIBLE.
  vi. $S \leftarrow S \cup \{\langle i^*, j^* \rangle\}, V_{pend} \leftarrow V_{pend} \setminus \{i^*\}$.
  vii. $\text{Actualizar contadores del proyecto } j^*$.
  viii. Comprobar si todos los proyectos cumplen L_j, si no, devolver INFEASIBLE.

(b) BRKGA:

- **Estructura:** $|ch| = n$ (un gen por voluntario).
- **Decoder:** Se ordenan los voluntarios según su valor en el cromosoma $ch[i]$. Siguiendo ese orden, cada voluntario elige el proyecto j que maximice $q(i, j, S)$ (la función greedy original).

Problema 2: Carga de Vehículos Eléctricos con Prioridad

Una estación tiene m cargadores y n vehículos esperando. Cada vehículo i necesita k_i kWh y tiene una importancia p_i. Los cargadores tienen diferentes potencias w_j. Existe una restricción de red: no pueden haber más de K vehículos de "Carga Rápida" (set R) cargando simultáneamente. El objetivo es maximizar la importancia total de los vehículos servidos.

Variables: $x_{ij} = 1$ si el vehículo i carga en el cargador j.

Heurística:

1. **Greedy:**
   - Define candidatos como pares $\langle vehículo, cargador \rangle$.
   - Diseña una función $q(i, j, S)$ que devuelva $-\infty$ si se viola la restricción de Carga Rápida y pondere la importancia p_i con la velocidad del cargador.
2. **BRKGA:**
   - Si el cromosoma tiene tamaño n, explica cómo el Decoder lo usaría para "perturbar" la importancia p_i de los vehículos antes de aplicar el Greedy.

Resolución Problema 2: Vehículos Eléctricos

Restricciones clave: Máximo K vehículos rápidos (R) simultáneos.

(a) Greedy:

- **Candidatos:** Pares $\langle vehículo_i, cargador_j \rangle$.
- **Función Greedy:**

$$q(i, j, S) = \begin{cases} -\infty & \text{si el cargador } j \text{ está ocupado} \\ -\infty & \text{si } i \in R \text{ y ya hay } K \text{ vehículos de } R \text{ cargando} \\ p_i \cdot w_j & \text{en otro caso} \end{cases}$$

- **Pseudocódigo:**
  i. Ordenar vehículos por importancia p_i (descendente).
  ii. Para cada vehículo i:
  iii. Encontrar cargador j que maximice $q(i, j, S)$.
  iv. Si existe j con $q > -\infty$, asignar y marcar cargador ocupado.
  v. Retornar asignación.

(b) BRKGA:

- **Estructura:** $|ch| = n$.
- **Decoder:** Modificamos la importancia del vehículo: $p'_i = p_i \cdot ch[i]$. Luego ejecutamos el Greedy con estos nuevos valores de p'_i.

Problema 3: Organización de un Festival de Cortometrajes

Tienes n cortos para proyectar en m salas. Cada corto i tiene una duración d_i y cada sala un tiempo máximo T_j. Hay dos tipos de cortos: Animación (A) y Ficción (F). Para mantener el interés, en cada sala, el número de cortos de animación debe ser **exactamente igual** al de ficción (o diferir en máximo 1 si el total es impar). El objetivo es maximizar el número total de cortos proyectados.

Heurística:

1. **Greedy:**
   - ¿Cómo manejarías la restricción de "equilibrio" entre A y F en la función greedy? (Pista: mira el examen de "First Dates" y cómo equilibran confesiones).
   - Escribe el pseudocódigo asegurando que no se exceda T_j.
2. **BRKGA:**
   - Define un cromosoma que ayude a decidir, cuando hay empate en la función greedy, qué corto elegir.

Resolución Problema 3: Festival de Cortometrajes

Restricciones clave: Tiempo T_j y equilibrio $|Animación - Ficción| \leq 1$.

(a) Greedy:

- **Candidatos:** Cortos $i \in \{A \cup F\}$ no asignados.
- **Función Greedy:**
  Sea $diff_j = |A_j| - |F_j|$ en la sala j.

$$q(i,j,S) = \begin{cases} -\infty & \text{si } d_i > \text{tiempo_restante}(j) \\ -\infty & \text{si } (i \in A \text{ y } diff_j \geq 1) \text{ o } (i \in F \text{ y } diff_j \leq -1) \\ 1 & \text{en otro caso} \end{cases}$$

- **Pseudocódigo:** Similar a los anteriores, pero iterando sala por sala hasta que no quepan más cortos o se agoten los candidatos que mantengan el equilibrio.

(b) BRKGA:

- **Estructura:** $|ch| = n$.
- **Decoder:** En el Greedy, cuando varios cortos tienen el mismo valor de $q(i,j,S) = 1$, el Decoder elige el que tenga el valor $ch[i]$ más alto (desempate aleatorio dirigido).

Problema 4: Almacenamiento de Productos Químicos

Un almacén tiene m estantes y n productos. Cada producto i tiene un peso w_i y cada estante una capacidad C_j. Algunos productos son "Inflamables" (set I). Por seguridad, no puede haber más de 2 productos inflamables en el mismo estante. Además, se quiere maximizar la "estabilidad" total, definida como la suma de los valores s_{ij} (estabilidad del producto i en el estante j).

Heurística:

1. **Greedy:**
   - Define candidatos y la función $q(i, j, S)$. La función debe penalizar fuertemente si ya hay 2 inflamables en el estante j.
   - Escribe el pseudocódigo.
2. **BRKGA:**
   - Diseña un esquema de cromosoma de tamaño $n \times m$. Explica cómo el Decoder usaría estos valores para modificar las estabilidades s_{ij} originales.

Resolución Problema 4: Productos Químicos

Restricciones clave: Máximo 2 inflamables (I) por estante.

(a) Greedy:

- **Candidatos:** Pares $\langle producto_i, estante_j \rangle$.
- **Función Greedy:**

$$q(i, j, S) = \begin{cases} -\infty & \text{si } w_i > \text{capacidad_restante}(j) \\ -\infty & \text{si } i \in I \text{ y } |P_j \cap I| \geq 2 \\ s_{ij} & \text{en otro caso} \end{cases}$$

- **Pseudocódigo:**
  i. Mientras queden productos:
  ii. $\langle i^*, j^* \rangle = \text{argmax } q(i, j, S)$ para todo i, j.
  iii. Si $q = -\infty$ terminar (no caben más).
  iv. Asignar i^* a j^* y actualizar capacidad y contador de inflamables.

(b) BRKGA:

- **Estructura:** $|ch| = n \times m$ (representando la afinidad producto-estante).
- **Decoder:** La estabilidad utilizada en el Greedy es $s'_{ij} = s_{ij} \cdot ch[i][j]$.

Problema 5: Planificación de Exámenes (Timetabling)

Hay n exámenes y m franjas horarias. Cada franja tiene una capacidad de alumnos C. Los exámenes i y k no pueden coincidir si comparten alumnos (matriz de conflicto $conf_{ik} = 1$). El objetivo es minimizar el número de franjas horarias utilizadas (o maximizar la ocupación de las primeras franjas).

Heurística:

1. **Greedy:**
   - Define una función greedy que elija el examen que sea "más difícil" de colocar (el que tenga más conflictos) y lo ponga en la mejor franja disponible.
2. **BRKGA:**
   - Estructura del cromosoma: $|ch| = n$.
   - Explica el Decoder: ¿Cómo convertirías los aleatorios en un orden de inserción de exámenes?

Resolución Problema 5: Planificación de Exámenes

Restricciones clave: No conflictos $conf_{ik}$ y capacidad C.

(a) Greedy:

- **Candidatos:** Exámenes i no planificados.
- **Función Greedy:**
  Utilizamos una estrategia de **Grado de Conflicto**: $q(i) = \sum_k conf_{ik}$ (cuantos más conflictos, más "difícil" es el examen y antes hay que ponerlo).
- **Pseudocódigo:**
  i. Calcular $q(i)$ para todos los exámenes.
  ii. Ordenar exámenes por $q(i)$ descendente.
  iii. Para cada examen i en ese orden:
  iv. Buscar la primera franja $t \in \{1..m\}$ donde no haya conflictos con exámenes ya puestos y quepan los alumnos.
  v. Si no hay franja, abrir una nueva o devolver error.

(b) BRKGA:

- **Estructura:** $|ch| = n$.
- **Decoder:** En lugar de usar $q(i)$ (el grado de conflicto), el orden de los exámenes lo determina el cromosoma (ordenar por $ch[i]$). La comprobación de conflictos y capacidad se mantiene igual.

Consejos para resolver las Heurísticas (Tips de examen):

1. **Candidatos:** Siempre suelen ser pares $\langle elemento, recurso \rangle$ (ej: $\langle alumno, clase \rangle$, $\langle corto, sala \rangle$).
2. **Función Greedy $q(\cdot)$:**
   - Si hay una restricción de **ratio** (ej: doble de buenos que de malos), haz que q devuelva un valor muy bajo (o $-\infty$) si asignar ese candidato rompe el ratio.
   - Si hay una restricción de **equilibrio** (ej: tantos hombres como mujeres), usa un multiplicador que favorezca al grupo que va "por detrás" en el conteo actual.
3. **Pseudocódigo:**
   - No olvides inicializar los conjuntos de "pendientes" y la solución vacía.
   - En cada iteración: calcular q para todos los candidatos, elegir el mejor, actualizar estados (capacidad restante, contadores de grupos) y quitar de pendientes.
4. **BRKGA Decoder:**
   - La respuesta estándar ganadora: "El cromosoma es un vector de tamaño n. El Decoder ordena los elementos de forma ascendente según el valor $ch[i]$ y luego aplica el algoritmo Greedy siguiendo ese orden".
   - Otra opción: "Se modifica la función greedy original: $q'(i,j) = q(i,j) \cdot ch[i]$".

Aquí tienes las resoluciones detalladas siguiendo el formato exacto de los exámenes que adjuntaste. Fíjate bien en cómo se redactan las funciones $q(\cdot)$ para que "guíen" a la solución hacia el cumplimiento de las restricciones.

¿Qué debes aprenderte de aquí para el examen?

1. **El uso del $-\infty$ o -1:** En la función greedy, úsalo para "matar" candidatos que violan restricciones. Es la forma más limpia de decir "esta opción no es legal".
2. **El Decoder de Ordenación:** Es el más común. Si te piden un BRKGA y no sabes qué hacer, di: *"El cromosoma define el orden de prioridad de los elementos; el Decoder recorre los elementos en ese orden y aplica la lógica Greedy para asignarlos"*.

3. **El Decoder de Sesgo (Bias):** Si el problema tiene una matriz de costes o beneficios (como a_{ij} o s_{ij}), di: *"El cromosoma modifica los pesos originales ($f'_{ij} = f_{ij} \cdot ch[i][j]$) permitiendo explorar soluciones distintas a la puramente codiciosa"*.

Para aprobar **Algorithmic Methods for Mathematical Models**, especialmente si ya dominas la Programación Lineal (LP), el secreto está en ser **muy riguroso con la notación** y entender los **patrones lógicos** que se repiten en el modelado y las heurísticas.

Aquí tienes los "Golden Tips" divididos por secciones:

1. El "Truco" de la Función Greedy ($q(\cdot)$)

En los exámenes, la función Greedy nunca es solo "elegir el máximo". Siempre tiene que **gestionar las restricciones difíciles** para evitar soluciones infactibles al final.

- **Usa el valor $-\infty$ (o $+\infty$ si minimizas):** Es la mejor forma de decir que un candidato es ilegal.
  - *Ejemplo:* Si una sala está llena, $q(i, j) = -\infty$.
- **Anticípate a las restricciones de "Ratio" o "Equilibrio":** Si el problema dice "tantos hombres como mujeres", tu función Greedy debe dar más puntos al sexo que vaya "perdiendo" en ese momento.
  - *Fórmula pro:* $q(i, j) = \text{beneficio} \times (1 + \text{necesidad_de_equilibrio})$.
- **Criterio de "El más difícil primero":** A veces, el Greedy no elige al "mejor" candidato, sino al "más conflictivo" para quitárselo de encima (como en el problema de los exámenes o el de las tareas con memoria).

2. BRKGA: No te compliques la vida

El 90% de las veces, el cromosoma en este examen se usa para una de estas dos cosas. Aprende estas frases de memoria:

- **Opción A (Orden de prioridad):** "El cromosoma es un vector de tamaño n (número de elementos). El Decoder ordena los elementos de forma ascendente según el valor de su gen $ch[i]$ y luego aplica el algoritmo Greedy siguiendo ese nuevo orden".
- **Opción B (Modificador de pesos/sesgo):** "El cromosoma tiene tamaño $n \times m$. El Decoder modifica la función de beneficio original multiplicándola por el gen correspondiente: $beneficio'_{ij} = beneficio_{ij} \times ch[i][j]$. Así, el Greedy explora zonas no óptimas".

3. Modelado: Cómo evitar que te quiten puntos

En las preguntas de "descripción informal de las restricciones", los profesores son muy estrictos:

- **PROHIBIDO:** "La suma de las variables x de i hasta n es menor o igual a 1". (Esto es describir la matemática).
- **PERMITIDO:** "Cada estudiante puede estar asignado, como máximo, a una sola clase". (Esto es describir el **significado físico**).
- **Restricciones de "solapamiento" (Overlap):** Si ves problemas de tiempos (bakery, tareas de computación), siempre vas a necesitar una variable binaria extra que diga: "¿El corto A va antes que el corto B?" (z_{ab}). Sin esa variable, no puedes evitar que dos cosas ocurran a la vez en la misma máquina.

4. Simplex y Branch & Bound (Rápido y Seguro)

- **¿Cuándo ramificar (branch)?** Solo si una variable que **debería ser entera** sale con decimales en la solución óptima del LP.
- **Costes reducidos (r_j):**
  - Si $r_j > 0$ (en MIN), la solución es óptima.
  - Si $r_j = 0$ para una variable no básica, hay infinitas soluciones óptimas.
- **Ratio Test:** No te equivoques aquí. Solo se dividen los valores de la columna b entre los valores **positivos** de la columna de la variable que entra. El menor ratio positivo gana.

5. Estrategia de Examen

1. **Asegura el Ejercicio 1 (Simplex):** Es mecánico. Hazlo despacio para no arrastrar un error de cálculo al valor de la función objetivo.
2. **Modelado (Ejercicio 2):** Define siempre qué significan tus variables antes de escribir las ecuaciones. Ejemplo: "$x_{ij} = 1$ si el objeto i va al contenedor j, 0 en caso contrario".
3. **Heurísticas (Ejercicio 3):** Si te bloqueas con la fórmula matemática de la $q(\cdot)$, **descríbela con palabras** primero. A veces dan puntos parciales si la lógica es buena aunque la fórmula esté mal escrita.
4. **Tiempo:** El examen de 3 horas suele sobrar si llevas bien la teoría, pero el modelado de "solapamientos" o "Big M" puede comerse mucho tiempo. Si te atascas en una restricción, sáltatela y sigue con la heurística.

Un último consejo: Fíjate en el examen de "First Dates" (PDF 1) y el de los alumnos (PDF 4). En ambos, la clave era el **equilibrio de grupos**. Si el problema que te ponen mañana tiene "tipos" de

elementos (hombres/mujeres, expertos/novatos, inflamables/estables), tu Greedy **debe** llevar contadores para saber cuántos hay de cada tipo en cada momento.

Para ir a por el 10, aquí tienes un desglose de las preguntas "trampa" que suelen caer sobre **Branch & Bound** y una explicación definitiva de los **Costes Reducidos**.

1. Preguntas Típicas de Branch & Bound (Más allá de lo básico)

Además de "cuándo ramificar", te pueden preguntar situaciones específicas sobre el árbol de búsqueda:

- **¿Cuándo podemos dejar de explorar un nodo (poda/pruning)?**
  i. **Por cota (Bound):** Si el valor del LP-Relax del nodo es **peor** (mayor en MIN, menor en MAX) que la mejor solución entera que ya conocemos.
  ii. **Por infactibilidad:** Si al añadir la restricción de ramificación ($x \leq 0$ o $x \geq 1$), el problema no tiene solución.
  iii. **Por optimalidad entera:** Si la solución del LP-Relax ya sale con todos los valores enteros. (Esa se convierte en nuestra nueva "Best Integer Solution").
- **¿Cuál es la cota (bound) de la solución óptima entera?**
  - Si estás en un problema de **MIN**, la solución del LP-Relax original es una **cota inferior (Lower Bound)**. La solución real entera nunca será mejor (menor) que esa.
  - Si estás en **MAX**, es una **cota superior**.
- **¿Qué variable elegir para ramificar?**
  - Suele elegirse la que tenga la **parte fraccionaria más cercana a 0.5** (la más "indecisa").
- **El impacto de las variables binarias:**
  - Si el problema es de variables 0-1 (binarias), recuerda que ramificar en x_i significa crear dos subproblemas: uno con $x_i = 0$ y otro con $x_i = 1$. No pongas $x_i \leq 0.4$, pon el valor entero directamente.

2. Costes Reducidos (r_j): Entiéndelo de una vez

El coste reducido de una variable **no básica** (las que valen 0) es, básicamente, el "precio" que tendrías que pagar por meter esa variable en la solución.

¿Qué significan según el signo? (Suponiendo que el problema es de MINIMIZAR):

- $r_j > 0$ **(Positivo):** Significa que si obligas a esa variable a valer 1, el valor total del coste **subirá** en esa cantidad. Por eso, si todos los r_j son positivos, estás en el **óptimo**, porque cualquier cambio aumentaría el coste.
- $r_j < 0$ **(Negativo):** Significa que si metes esa variable en la solución, el coste total **bajará**. Por eso, en el algoritmo Simplex, elegimos la variable con el coste reducido más negativo para que entre en la base.
- $r_j = 0$**:**
  - Si la variable es **básica**, siempre es 0.
  - Si la variable es **NO básica** y su $r_j = 0$, ¡CUIDADO! Significa que hay **múltiples soluciones óptimas**. Puedes meter esa variable en la base sin que el valor de la función objetivo cambie.

¿Cómo se calculan a mano rápido?

Si te dan la función objetivo $z = 2x_0 + 3x_1 + 0x_2$ y en el óptimo x_1 y x_2 son básicas:

1. Despejas las variables básicas de las restricciones.
2. Sustituyes esos "chorizos" en la función objetivo.
3. Al final te quedará algo como: $z = 15 + (4)x_0 + (0)x_1 + (0)x_2$.
4. El número que acompaña a la x_0 (que es la no básica) es su **coste reducido**.

3. Otros conceptos "Satélite" que pueden caer

- **Precios Sombra (Shadow Prices):**
  - Es lo que mejorarías la función objetivo si pudieras aumentar en 1 unidad el lado derecho (b) de una restricción (ej: tener 1 kg más de materia prima).
  - Pregunta típica: "¿Cuánto pagarías como máximo por una unidad extra de recurso?". La respuesta es el precio sombra.
- **Infactibilidad vs No Acotado (Unbounded):**
  - **No acotado:** Si en una columna del Simplex intentas hacer el *ratio test* y todos los números son ≤ 0. Significa que la variable puede crecer hasta el infinito y tu beneficio también.
  - **Infactible:** Si después de hacer todo el Simplex, alguna "variable artificial" (las que se ponen con la M grande) sigue valiendo algo distinto de cero.
- **Degeneración:**

- Ocurre cuando en la columna del valor de las variables (b), aparece un **0**. Esto puede causar que el Simplex entre en un bucle infinito (ciclado).

4. Resumen final de Heurísticas (El patrón de oro)

Si mañana te bloqueas diseñando una heurística, sigue este esquema mental:

1. **Estado:** ¿Qué he decidido ya? (Ej: "Alumnos ya asignados").
2. **Candidatos:** ¿Qué decisiones puedo tomar ahora? (Ej: "Parejas alumno-clase que no violan capacidad").
3. **Evaluación (q):** De los candidatos, ¿cuál parece mejor **ahora** pero también **ayuda a que no se rompa nada luego**?
   - Si el recurso es escaso $\rightarrow$ penaliza usarlo.
   - Si hay que cumplir un mínimo $\rightarrow$ premia acercarte al mínimo.
   - Si hay conflicto $\rightarrow$ prioriza colocar lo que más conflictos genera primero.

¡Mucha suerte! Si dominas el Simplex y el modelado, tienes más de medio examen en el bolsillo. Lo de las heurísticas es echarle imaginación y escribirlo con orden.